



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

GCC PLUGIN FOR STATIC ANALYZER SUPPORT

GCC PLUGIN PRO STATICKÉ ANALYZÁTORY

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

LUKÁŠ PŠEJA

SUPERVISOR

VEDOUCÍ PRÁCE

Dr. Ing. PETR PERINGER,

BRNO 2026

Bachelor's Thesis Assignment



172017

Institut: Department of Intelligent Systems (DITS)
Student: **Pšeja Lukáš**
Programme: Information Technology
Title: **GCC plugin for static analyzer support**
Category: Software analysis and testing
Academic year: 2025/26

Assignment:

1. Get acquainted with *CodeListener* component from the *Predator* project. Analyze the implementation of *CodeListener* and similar projects. Additionally, study the relevant data structure visualization concepts.
2. Design a new GCC plugin as a modern alternative to *CodeListener* and add new functionality (built-in visualization support and export in JSON format). Focus on improvement of the plugin interface.
3. Implement the plugin in C++ along with appropriate examples for demonstration purposes. Verify the plugin's compatibility by using it with *Predator*.
4. Test the plugin using the created examples with various GCC versions. Evaluate obtained results.

Literature:

- GitHub repository of Predator project.
<https://github.com/kdudka/predator>

Requirements for the semestral defence:

- First two points of assignment.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Peringer Petr, Dr. Ing.**
Head of Department: Kočí Radek, Ing., Ph.D.
Beginning of work: 1.11.2025
Submission deadline: 13.5.2026
Approval date: 3.11.2025

Abstract

This bachelor's thesis deals with the analysis of the Code Listener infrastructure, which serves as the foundation for creating static analyzers, and its subsequent refactoring into a new plugin.

Abstrakt

Tato bakalářská práce se zabývá analýzou infrastruktury Code Listener, která poskytuje základ pro tvorbu statických analyzátorů zdrojového kódu, a její následnou refaktorizací do nového pluginu.

Keywords

Code Listener infrastructure, GCC, LLVM, plugin, refactoring, static analysis, C, C++

Klíčová slova

infrastruktura Code Listener, GCC, LLVM, plugin, refaktorování, statická analýza, C, C++

Reference

PŠEJA, Lukáš. *GCC plugin for static analyzer support*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Dr. Ing. Petr Peringer,

GCC plugin for static analyzer support

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Dr. Ing. Petr Peringer. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used Grammarly to correct spelling and other language mistakes.

.....

Lukáš Pšeja
April 22, 2026

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Thesis structure	4
2	State of the Art	5
2.1	Overview	5
2.2	GCC and the Plug-in Interface	5
2.3	Architecture of the Original Code Listener	7
2.4	Mandatory Pre-Analysis Steps	9
2.5	Existing Adapters and Extensions	9
2.6	Limitations of the Existing Architecture	9
3	Design	11
4	Implementation	12
5	Testing and Evaluation	13
6	Conclusion	14
	Bibliography	15
A	Build and Usage Instructions	16

List of Figures

- 2.1 Block diagram of the original Code Listener infrastructure (reproduced from [1]).
Data flows left to right: adapters translate each compiler's internal IR into a
stream of listener callbacks. Filters and listeners process that stream. Code
storage accumulates the result into a persistent object model. 7

Chapter 1

Introduction

This bachelor’s thesis focuses on the design and implementation of a modernized reimplementation of the Code Listener infrastructure, a C++ library that serves as a vital bridge between compilers and static analysis tools, particularly the Predator shape analysis tool suite. The original Code Listener architecture, while successful in enabling static analysis of C programs that manipulate heap memory, exhibits several design limitations that have accumulated over time. This thesis aims to address these constraints by replacing the callback-driven pipeline with a persistent Code Model, demoting analyses from hard-wired pre-analysis steps to optional, on-demand annotation services, and introducing a structured query interface and uniform support for exporting results in common formats.

1.1 Motivation

Software development is a complex and error-prone process, especially as software systems grow in size and complexity. As software systems evolve, certain categories of defects become undetectable solely through testing, as test suites can only evaluate explicitly covered execution paths, leaving much of the program’s state space unexplored.

Nowhere is this limitation more critical than in low-level system programming languages such as C and C++, where developers bear the burden of manual memory management. In these environments, insidious bugs such as memory leaks, buffer overflows, use-after-free errors, and dangling pointer dereferences frequently hide within unexplored execution paths. The consequences of these hidden defects can be catastrophic, particularly in mission-critical domains like aerospace engineering. While rigorous testing prevents many fatal accidents, memory mismanagement has repeatedly compromised modern aircraft. For instance, regulatory agencies have had to issue mandatory airworthiness directives for both the Boeing 787 and the Airbus A350, requiring airlines to periodically reboot the aircraft to clear memory buffers and prevent the total mid-flight loss of critical systems. These near-misses underscore a fundamental reality: traditional dynamic testing is insufficient to guarantee memory safety in high-stakes environments.

To prevent such failures, developers must rely on advanced static analysis tools, such as the Predator shape analysis suite, which is capable of formally verifying a program’s state space without executing it. However, developing and maintaining these analyzers presents a significant architectural challenge: extracting the program’s semantics from the compiler’s internal representation. Analyzers must interface with compilers to understand the code, but native compiler APIs are notoriously complex, tightly coupled to specific versions, and

largely undocumented. Building analysis tools directly on top of these raw APIs leads to brittle, maintenance-heavy software.

To overcome this bottleneck, a clean abstraction layer is required. Specifically, researchers need a modular middleware that can abstract away the massive, often undocumented codebases of industrial compilers, providing instead a concise, unified, and well-documented object-oriented API. This ensures that static analysis tools can be developed independently of the underlying compiler architecture.

1.2 Thesis structure

The remainder of this thesis is structured as follows. Chapter 2 provides a deeper technical analysis of the legacy Code Listener architecture. Chapter 3 outlines the architectural modernization, detailing the transition from a callback-driven pipeline to a persistent Code Model. Chapter 4 documents the implementation process, including development environment setup, portability considerations, and code metrics regarding the refactored infrastructure. Chapter 5 presents the methodology and results of regression testing, demonstrating the correctness and performance of the new implementation against the existing test suite. Finally, Chapter 6 summarizes the contributions of this work and outlines potential avenues for future research and development.

Chapter 2

State of the Art

This chapter provides a detailed account of the existing Code Listener infrastructure in the Predator repository [6, 5] and as described in the original publication [1]. Understanding the design decisions and limitations of the current implementation is necessary background for the design choices made in this thesis.

2.1 Overview

The Code Listener infrastructure was originally developed at the Faculty of Information Technology, Brno University of Technology, as a support library for the Predator and Forester research prototype analyzers [1]. Its central goal is to *decouple* the static analysis logic from the internals of particular compilers. Compilers such as GCC expose rich intermediate representations (IRs), but their internal APIs are large, poorly documented in places, and subject to change across versions. Code Listener provides a concise, well-documented C++ API that absorbs this complexity and exposes a stable interface to analyzer authors.

From the perspective of an analyzer, Code Listener behaves like a black box: the analyzer registers itself, a *GCC plug-in* is loaded, and when GCC finishes compiling a translation unit, the analyzer finds a fully populated *code storage* object ready to inspect. The analyzer never needs to call a GCC API directly.

2.2 GCC and the Plug-in Interface

GCC is a collection of compilers for C, C++, Fortran, Ada, Go, and other languages, organized around a three-phase design. A *front end* that parses source code and produces an abstract syntax tree (AST), a *middle end* that performs language-independent optimizations on several IR levels, and a *back end* that generates machine code [2]. The plug-in interface [3] allows client code to register callbacks at well-defined points in the compilation pipeline, such as after parsing, after each optimization pass, after each function is compiled, and so on.

Code Listener registers a callback that fires once per function. This occurs after the front end produces the function’s GIMPLE IR, but before the optimizer runs significant transformations. At this stage, the IR is in *low-level GIMPLE* form. This form uses a three-address representation, where expressions become sequences of simple assignments, unary and binary operations, calls, and control-flow instructions.

2.2.1 GIMPLE Intermediate Representation

GIMPLE is the principal IR used by GCC’s middle end [2]. In GIMPLE, each function is represented as a *control flow graph* (CFG) whose nodes are *basic blocks*, sequences of instructions that execute without intervening branches. Edges represent possible transitions between blocks. Instructions in low-level GIMPLE are restricted to a three-address form. Each instruction reads at most two operands and writes at most one result. Compound expressions are split into sequences by introducing temporary variables so as not to break the three-address form.

GIMPLE recognizes, and Code Listener exposes the following instruction kinds:

- UNOP** A unary operation of the form $dst := \circ src$, where $\circ$ is the identity (assignment), logical not, bitwise not, or unary minus.
- BINOP** A binary operation $dst := src_1 \circ src_2$ supporting comparison, arithmetic (including pointer arithmetic), logical, and bitwise operators.
- CALL** A function call $dst := f(arg_1, \dots, arg_n)$, where f may be a statically known function, an external function, or a function pointer. A **CALL** is treated as a *non-terminal* instruction, which means that it may appear in the middle of a basic block.
- JMP** An unconditional branch to a single target block, which is always terminal.
- COND** A conditional branch with a Boolean operand and two targets (*then* and *else*), also always terminal.
- SWITCH** A generalization of **COND** for integer or enumerated operands, specifying a list of value–target pairs and a default target; always terminal.
- RET** A function return carrying an optional return value, always terminal, marks an end-point of the CFG.
- ABORT** An unconditional program termination, inserted after calls to **noreturn** functions such as `abort()` from `<stdlib.h>`, which is also always terminal.

2.2.2 Operands and Accessors

The key structural element of both the GCC adapter API and the code storage API is the *operand*, represented by the C structure `cl_operand`. An operand is either void (no value), a constant (`cl_cst`), or a variable (`cl_var`). Constants cover numeric literals, string literals, and function addresses. Variables carry a scope tag (global, static, or local) and a unique integer identifier (*uid*) that is globally unique within a translation unit when using the GCC front end.

Operands are augmented by a chain of *accessors* (`cl_accessor`) that modify the semantics of the base operand [1]:

- **Reference** (`CL_ACCESSOR_REF`): take the address of the operand.
- **Dereference** (`CL_ACCESSOR_DEREF`): load the value at the pointer.
- **Array item** (`CL_ACCESSOR_DEREF_ARRAY`): index into an array.
- **Struct item** (`CL_ACCESSOR_ITEM`): access a field of a composite type.

The accessor chain is subject to structural constraints: a dereference, if present, must appear at the beginning of the chain, and a reference, if present, must appear at the end. Chained dereferences are forbidden. If a C expression would require multiple pointer indirections in one operand, Code Listener splits it into a sequence of instructions, each with at most one dereference. This design keeps the instruction set small while covering all C language pointer and aggregate semantics.

2.2.3 Type System

Since C is statically typed, every operand and accessor carries a reference to a `cl_type` structure that describes the compile-time type. A `cl_type` is characterized by its *kind* (integer, floating-point, pointer, array, struct, union, function type, void, etc.) and, for non-atomic types, by an array of `cl_type_item` references that describe the constituent sub-types or fields. Struct and union items additionally carry a byte *offset* from the start of the containing object. All types, variables, and functions are assigned globally unique long integer uids, which Code Listener exposes for use as dictionary keys or for identifying entities across compilation units.

2.3 Architecture of the Original Code Listener

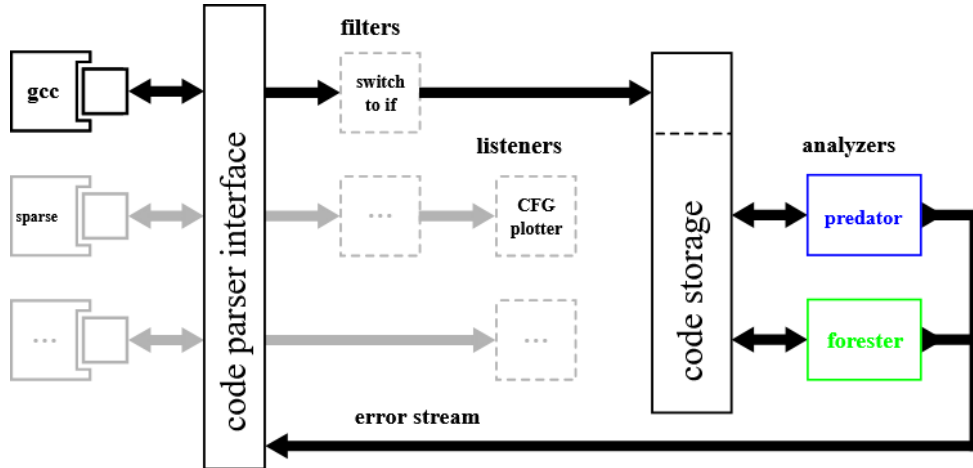


Figure 2.1: Block diagram of the original Code Listener infrastructure (reproduced from [1]). Data flows left to right: adapters translate each compiler’s internal IR into a stream of listener callbacks. Filters and listeners process that stream. Code storage accumulates the result into a persistent object model.

Figure 2.1 shows the high-level structure of the original Code Listener. Three conceptual layers are present: the *code parser interface*, the *filter and listener pipelines*, and the *code storage*.

2.3.1 Code Parser Interface and Adapters

The *code parser interface* is specified entirely in plain C (the header `cl/code_listener.h`) so that it can be consumed by code parsers written in C, for example, Sparse, the Linux kernel’s semantic analysis tool, which uses certain C++ keywords as identifiers in its own

headers [1]. The interface consists of a set of callback function pointers that the adapter calls in a prescribed order as it traverses the compiler’s internal data structures.

An *adapter* is a small component embedded inside each supported compiler. It is responsible for translating the compiler-specific IR into a sequence of calls to the Code Listener API. Two adapters were shipped with the original distribution: one for GCC (the primary adapter, supporting both C and C++ [4]) and one for Sparse. A third adapter for Clang/LLVM was contributed as a bachelor’s thesis [8].

The adapter invokes the callbacks in a well-defined order corresponding to a depth-first traversal of the program. Specifically, it opens a namespace, a type, a function, a basic block, emits instructions one by one, closes the block, closes the function, closes the namespace, and so forth. The listener on the other side of the API can therefore reconstruct the complete program structure by maintaining a stack.

2.3.2 Filters

A *filter* is a component that sits between an adapter and a downstream listener. It implements the same callback interface on both its input and its output, intercepts the incoming stream of events, optionally transforms them, and re-emits them downstream. Filters can be chained. The original distribution ships one filter that is unconditionally active in all predefined configurations: the *switch-to-if* filter, which translates every **SWITCH** instruction into an equivalent sequence of **COND** instructions. This means that the code storage and all analyzers built on it never observe **SWITCH** instructions, regardless of whether the original source code contained any.

2.3.3 Listeners

Beyond code storage, Code Listener also exposes the listener interface to diagnostic tools. A *listener* uses the same input-side callback API but does not re-emit the stream; instead, it processes it in place. The original distribution includes a CFG plotter (`cl_dotgen`) that generates Graphviz visualizations of control flow graphs, and an intermediate code printer (`cl_pp`) that produces a textual dump of the three-address IR.

2.3.4 Code Storage

The *code storage* is a special listener that, rather than processing the stream on the fly, accumulates the incoming events into a persistent, navigable object model. Once the entire program has been pushed through the pipeline, an analyzer can query this model at will.

The top-level object is named **Storage**. It contains three lookup containers: **TypeDb** (all types encountered in the program), **VarDb** (all variables), and **FncDb** (all functions). Each function is represented by a **Fnc** value that provides the function’s argument list, its local variable list, and its CFG. The CFG is a **ControlFlow** object that can be iterated as a collection of **Block** values, each of which holds an ordered sequence of **Insn** (instruction) values. Instructions store their operands and branch targets in `std::vector` containers. Every significant object, that being a type, variable, function, or instruction, carries a `cl_loc` location structure that records the corresponding source file name, line number, and column, so that an analyzer can report errors in the standard GCC diagnostic format.

2.4 Mandatory Pre-Analysis Steps

Before handing control to the registered analyzer, the existing Code Listener infrastructure unconditionally executes four analysis passes that further transform or annotate the code storage:

1. **Call graph construction.** A call graph is built by scanning all `CALL` instructions in the code storage. The graph is then made available through the `Storage` object.
2. **Loop-closing edge annotation.** The CFG of each function is examined to identify *back edges*, meaning the edges that lead to a previously-visited basic block, and these are annotated to assist analyses that need to distinguish loop-entry points from other join points.
3. **Points-to analysis.** A flow-insensitive, context-insensitive points-to analysis is run over the code storage to compute approximate pointer target sets for each variable.
4. **Liveness analysis.** A standard backward liveness analysis is computed per function. Each instruction in the code storage is extended with a bit vector recording which variables can be killed at that point.

All four passes are non-optional. They execute for every invocation of Code Listener, regardless of whether the registered analyzer actually uses their results. There is no mechanism for an analyzer to opt out of a pass or to request a different analysis in its place.

2.5 Existing Adapters and Extensions

Since its initial release, Code Listener has been extended by several student projects at FIT BUT:

- **Sparse adapter** (master’s thesis, Jan Pokorný, 2012) [7]: an adapter for the Sparse semantic checker used in Linux kernel development. Sparse is written in C and serves as a lightweight alternative front end.
- **LLVM/Clang adapter** (bachelor’s thesis, Veronika Šoková, 2014) [8]: an adapter that hooks into the Clang/LLVM compilation pipeline and feeds the LLVM IR into the Code Listener callback interface. This adapter was later integrated into the main Predator distribution as an official alternative front end.
- **C++ support for the GCC adapter** (bachelor’s thesis, David Kašpar, 2014/2015) [4]: an extension of the GCC adapter that handles GCC’s representation of C++ constructs, these being virtual methods, destructors, and name mangling, so that Predator and Forester can analyze C++ programs.

2.6 Limitations of the Existing Architecture

The original Code Listener architecture was a significant practical achievement when it was designed. It successfully decoupled Predator and Forester from GCC, enabled the LLVM and Sparse adapters, and supported analysis of real industrial code. However, as the tool suite has grown, several structural limitations have become apparent:

Mandatory, non-optional transformations. The switch-to-if filter always executes. An analyzer that directly supports SWITCH instructions cannot accept the unmodified IR.

Mandatory, non-optional pre-analyses. The four analyses described in Section 2.4 always run, consuming time and memory even when the registered analyzer does not need results. There is no partial evaluation or lazy initialization.

No standardized annotation mechanism. Analyzers that wish to attach their own data to code model entities must maintain their own side tables with no infrastructure support.

No structured query API. Iterating over the code model requires direct traversal through nested C++ containers. There is no abstraction for common access patterns.

No uniform export interface. Producing output in structured formats (JSON, DOT) requires ad-hoc code in each analyzer.

GCC version coupling. The adapter code uses several GCC-internal APIs that change across major GCC versions. Upgrading the GCC toolchain has historically required patching the adapter [5].

These limitations motivate the redesign described in the following chapter.

Chapter 3

Design

Chapter 4

Implementation

Chapter 5

Testing and Evaluation

Chapter 6

Conclusion

Bibliography

- [1] DUDKA, K.; PERINGER, P. and VOJNAR, T. An Easy to Use Infrastructure for Building Static Analysis Tools. *Lecture Notes in Computer Science*, 2012, vol. 2012, no. 6927, p. 527–534. ISSN 0302-9743. Available at: <http://www.springerlink.com/content/75024011tk386572/>.
- [2] *GCC Internals* online. Available at: <https://gcc.gnu.org/onlinedocs/gccint/>. [cit. 2026-04-04].
- [3] *GCC Plugin API* online. Available at: <https://gcc.gnu.org/onlinedocs/gccint/Plugins.html>. [cit. 2026-04-04].
- [4] KAŠPAR, D. *Extension of the Code Listener Infrastructure Adding C++ Support*. Brno, Czech Republic, 2015. Bachelor’s Thesis. Brno University of Technology, Faculty of Information Technology.
- [5] PERINGER, P.; ŠOKOVÁ, V. and VOJNAR, T. PredatorHP Revamped (Not Only) for Interval-Sized Memory Regions and Memory Reallocation (Competition Contribution). In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2020), Part II*. Springer, 2020, vol. 12079, p. article, Competition Contribution. Lecture Notes in Computer Science.
- [6] PERINGER, P.; ŠOKOVÁ, V.; TRTÍK, M.; VOJNAR, T.; HOLÍK, L. et al. Predator Shape Analysis Tool Suite. In: *Proceedings of HVC 2016*. Zurich: Springer International Publishing, 2016, vol. 10028, p. 202–209. Lecture Notes in Computer Science. ISBN 978-3-319-49052-6. Available at: http://link.springer.com/chapter/10.1007/978-3-319-49052-6_13.
- [7] POKORNÝ, J. *Creation of Sparse Adapter for the Code Listener Infrastructure*. Brno, Czech Republic, 2012. Master’s Thesis. Brno University of Technology, Faculty of Information Technology.
- [8] ŠOKOVÁ, V. *Development of an LLVM Adapter for the Code Listener Infrastructure*. Brno, Czech Republic, 2014. Bachelor’s Thesis. Brno University of Technology, Faculty of Information Technology.

Appendix A

Build and Usage Instructions

Requirements

- GCC 12 (gcc-12, g++-12, gcc-12-plugin-dev)
- CMake 3.28+
- patch (used to apply the Predator shim patch)
- Internet access at configure time (nlohmann/json fetched via `FetchContent`)

On Debian/Ubuntu:

```
sudo apt install gcc-12 g++-12 gcc-12-plugin-dev cmake patch
```

Clone

```
git clone --recurse-submodules \
  git@github.com:pseja/gcc-plugin-for-static-analyzer-support.git
cd gcc-plugin-for-static-analyzer-support
```

Configure and Build

```
make configure # cmake -B build -DTARGET_GCC=gcc-12
make build     # cmake --build build -j$(nproc)
```

Key build outputs in build/:

- `libcl_gcc.so` — the GCC plugin
- `libcl_json_dump.so` — JSON dump analyzer
- `libcl_callgraph_dot.so` — call-graph DOT analyzer
- `libcl_recursion_check.so` — recursion detector
- `cl_merge`, `cl_analyze` — standalone tools
- `analyzers/predator/sl_build/libsl_analyzer.so` — Predator (if `WITH_PREDATOR=ON`)

Run Tests

```
make test
```

Run an Analyzer (Single TU)

```
gcc-12 -S file.c -o /dev/null \  
-fplugin=./build/libcl_gcc.so \  
-fplugin-arg-libcl_gcc-load-analyzer=./build/libcl_json_dump.so \  
-fplugin-arg-libcl_gcc-args=file.json
```

Whole-Program Analysis

```
# 1. compile each translation unit to a JSON model  
gcc-12 -S a.c -o /dev/null -fplugin=./build/libcl_gcc.so \  
-fplugin-arg-libcl_gcc-load-analyzer=./build/libcl_json_dump.so \  
-fplugin-arg-libcl_gcc-args=a.json  
# ... repeat for b.c -> b.json, c.c -> c.json  
  
# 2. merge  
./build/cl_merge a.json b.json c.json -o merged.json  
  
# 3. analyze  
./build/cl_analyze merged.json \  
--analyzer=./build/libcl_callgraph_dot.so \  
--args=callgraph.dot
```